SIMATS ENGINEERING

ASSESSMENT TOOL 1: AI Model Design Exercise

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA01**

COURSE OUTCOME COVERED

CO4: *Develop intelligent software agents using suitable agent architectures and learning techniques.*(BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 1

Annexure A

AI Model Design Exercise

Code of Conduct:

I S.Venkateswarlu_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **S.Venkateswarlu**

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Dataset Selection and Data Understanding	10%	
3. Data Preprocessing and Feature Engineering	10%	
4. AI Technique Selection and Justification	10%	
5. Model Design / Architecture	10%	
6. Algorithm / Pseudocode Development	5%	
7. Model Implementation and GitHub Repository	15%	
8. Experimental Design and Results	10%	
9. Performance Evaluation and Metrics	10%	
10. Result Analysis and Interpretation	5%	
11. Limitations and Future Enhancements	5%	
12. Documentation and Technical Presentation	10%	
Total	100%	

Signature of the Faculty Total Marks Awarded

STUDENT PERFORMANCE PREDICTION USING INDUCTIVE LEARNING

1. Problem Statement

A university wants to predict whether a student is likely to **Pass or Fail** a course based on academic and behavioral factors such as attendance, internal marks, assignment performance, study hours, and previous academic performance.

The system uses historical student records as training examples and learns general patterns from these examples. The learned model is then used to predict the performance of new students.

Input Features:

- Attendance percentage
- Internal marks
- Assignment performance
- Study hours per day
- Previous academic performance

Output:

- Pass
- Fail

AI Task: Classification using inductive learning.

2. Objectives

The main objectives are:

1. To collect a dataset containing student academic information.
2. To preprocess the student data.
3. To identify important features affecting student performance.
4. To develop an inductive learning model.
5. To learn generalized decision rules from training examples.
6. To predict whether a student will pass or fail.
7. To evaluate the model using suitable performance metrics.
8. To analyze the strengths and limitations of the developed system.

3. Dataset Description

Dataset

A student performance dataset is used for this project.

Feature	Data Type	Description
Attendance	Numeric	Percentage of classes attended
Internal_Marks	Numeric	Internal examination marks
Assignment_Score	Numeric	Assignment performance
Study_Hours	Numeric	Average study hours per day
Previous_Marks	Numeric	Marks obtained in previous examination
Result	Categorical	Pass or Fail

Sample Dataset

Attendance (%)	Internal Marks	Assignment Score	Study Hours	Previous Marks	Result
90	85	90	4	82	Pass
85	78	85	3	75	Pass
60	45	50	1	48	Fail
95	92	95	5	90	Pass
70	55	60	2	58	Fail
88	80	82	4	79	Pass
65	48	55	1	50	Fail
92	86	88	4	84	Pass
75	65	70	2	68	Pass
55	40	45	1	42	Fail

Target Variable: Result

Classification Classes: Pass and Fail

4. Data Preprocessing

The following preprocessing steps are performed:

Step 1: Data Cleaning

Check the dataset for:

- Missing values
- Duplicate records
- Invalid marks
- Incorrect attendance values

Step 2: Missing-Value Handling

If numerical values are missing, they can be replaced using the mean or median.

Step 3: Feature Encoding

The target variable is converted into numerical form:

- Pass → 1
- Fail → 0

Step 4: Feature Selection

The following features are selected:

- Attendance
- Internal Marks
- Assignment Score
- Study Hours
- Previous Marks

Step 5: Train-Test Split

The dataset is divided into:

- **80% Training Data**
- **20% Testing Data**

The training data is used to learn patterns, while the testing data evaluates generalization to unseen students.

5. Selected AI Technique and Justification

Inductive Learning

Inductive learning learns general rules from specific examples.

For example:

Training examples:

- High attendance + high marks → Pass
- Low attendance + low marks → Fail
- High study hours + good previous marks → Pass

The algorithm identifies common patterns and generates generalized rules.

Why Inductive Learning?

Inductive learning is suitable because:

1. Historical student records are available.
2. The target is a classification problem.
3. The model can learn patterns from examples.
4. The learned rules can be used for new students.
5. The resulting rules can be interpreted easily.

A **Decision Tree classifier** can be used as the practical implementation of inductive learning.

6. Model Design / Architecture

Model Flow

Student Dataset

↓

Data Cleaning

↓

Feature Selection

↓

Train-Test Split

↓

Inductive Learning

↓

Decision Tree Model

↓

Learn Generalized Rules

↓

New Student Data

↓

Prediction

↓

PASS / FAIL

Model Inputs

Attendance

Internal Marks

Assignment Score

Study Hours

Previous Marks

Model Output

PASS

or

FAIL

Example Learned Rules

The model may learn rules such as:

IF Attendance >= 75

AND Internal Marks >= 50

THEN Result = PASS

IF Attendance < 60

AND Internal Marks < 50

THEN Result = FAIL

IF Study Hours >= 3

AND Previous Marks >= 60

THEN Result = PASS

These are examples of generalized rules; the actual rules should be obtained from the trained model.

7. Algorithm / Pseudocode

Algorithm: Student Performance Prediction

Input: Student dataset D

Output: Predicted student result

1. Start.
2. Load the student dataset.
3. Identify input features and target variable.
4. Check for missing and invalid values.
5. Clean and preprocess the dataset.
6. Encode the target variable.
7. Divide the dataset into training and testing sets.
8. Initialize the inductive learning model.
9. Train the model using the training examples.
10. Generate generalized decision rules.
11. Provide test student data to the trained model.

12. Predict Pass or Fail.
13. Calculate accuracy, precision, recall and F1-score.
14. Display the results.
15. Stop.

8. Implementation

Programming Language

Python

Libraries

- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- Seaborn

Python Implementation

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    ConfusionMatrixDisplay
)

# Load dataset
data = pd.read_csv("student_performance.csv")

# Features
X = data[
```

```
    [
        "Attendance",
        "Internal_Marks",
        "Assignment_Score",
        "Study_Hours",
        "Previous_Marks"
    ]
]

# Target
y = data["Result"]

# Convert Pass/Fail into numerical values
y = y.map({"Fail": 0, "Pass": 1})

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Create model
model = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=4,
    random_state=42
)

# Train model
model.fit(X_train, y_train)
```

```
# Prediction
y_pred = model.predict(X_test)

# Performance metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Fail", "Pass"]
)

disp.plot()
plt.title("Student Performance Confusion Matrix")
plt.show()

# Predict a new student
new_student = [[85, 75, 80, 3, 78]]

prediction = model.predict(new_student)

if prediction[0] == 1:
    print("Predicted Result: PASS")
else:
```

```
print("Predicted Result: FAIL")
```

9. Experimental Results

The trained model is tested using previously unseen student records.

The following table can be included in the report after executing the program:

Metric	Result
Accuracy	XX%
Precision	XX%
Recall	XX%
F1-Score	XX%

Important: Replace XX% with the actual values obtained when you run the program. Do not invent experimental results.

Sample Prediction

For a student with:

- Attendance = 85%
- Internal Marks = 75
- Assignment Score = 80
- Study Hours = 3
- Previous Marks = 78

the model can predict:

Result = PASS

10. Performance Evaluation

The following metrics are used.

Accuracy

Accuracy measures the percentage of correctly classified students.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

Precision measures how many students predicted as Pass actually passed.

$$Precision = \frac{TP}{TP + FP}$$

Recall

Recall measures how many actual Pass students were correctly identified.

$$Recall = \frac{TP}{TP + FN}$$

F1-Score

F1-score is the harmonic mean of precision and recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

A confusion matrix is also used to understand correct and incorrect predictions.

11. Result Analysis

The inductive learning model learns patterns from previous student examples and applies those patterns to unseen students.

Generally, students with:

- Higher attendance
- Better internal marks
- Higher assignment scores
- More study hours
- Better previous academic performance

are more likely to be classified as **Pass**.

Students having poor performance across several features are more likely to be classified as **Fail**.

The decision tree also provides an interpretable representation of the learned decision rules, making it easier for teachers and administrators to understand the factors influencing predictions.

12. Limitations

1. The model depends heavily on the quality of the training dataset.
2. A small dataset may not represent all students.
3. Student performance can be affected by factors not included in the dataset.
4. Decision trees can overfit if they are allowed to become excessively deep.
5. Historical academic patterns may not always predict future performance accurately.
6. The model should support educational decisions rather than completely replace human judgment.

13. Future Enhancements

The system can be improved by:

1. Collecting a larger student dataset.
2. Including more features such as participation and learning behavior.
3. Applying feature engineering.
4. Using hyperparameter optimization.
5. Comparing Decision Tree with Random Forest, SVM and Neural Networks.
6. Developing a web-based prediction system.
7. Adding early-warning notifications for students at risk of failing.
8. Using explainable AI techniques to show why a student received a particular prediction.
9. Periodically retraining the model using new academic data.

14. Conclusion

This project demonstrates how **Inductive Learning** can be applied to student performance prediction. Historical student records containing attendance, internal marks, assignment performance, study hours and previous marks are used as training examples.

A Decision Tree is implemented as the practical inductive learning model. The model learns generalized patterns from training examples and predicts whether a new student is likely to pass or fail.

Performance is evaluated using accuracy, precision, recall, F1-score and a confusion matrix. The approach is simple, interpretable and suitable for an academic student-performance prediction system.

Overall, the proposed system can help universities identify students who may require additional academic support at an early stage.

15. References

1. Mitchell, T. M., *Machine Learning*, McGraw-Hill, 2024.
2. Quinlan, J. R., *C4.5: Programs for Machine Learning*, Morgan Kaufma 2023.
3. Géron, A., *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, O'Reilly Media.
4. Pedregosa, F., et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, 2025.
5. Scikit-learn Documentation – Decision Tree Classifier.
6. Pandas Documentation – Data Analysis and Manipulation.

7. UCI Machine Learning Repository – Student Performance Dataset.

16. GitHub Repository Link

Create a public GitHub repository containing:

student-performance-prediction/

```
│
├── student_performance.csv
├── student_performance.py
├── README.md
├── requirements.txt
├── results/
│   ├── confusion_matrix.png
│   └── results.txt
└── report/
    └── Student_Performance_Report.pdf
```